

ИТОГОВЫЙ ПРОЕКТ · УСТРОЙСТВО КОДА

Путеводитель по коду

Какой класс за что отвечает и что означает каждая важная переменная
— от графа города до таблицы DP

Планировщик маршрута курьера · C# · .NET 10

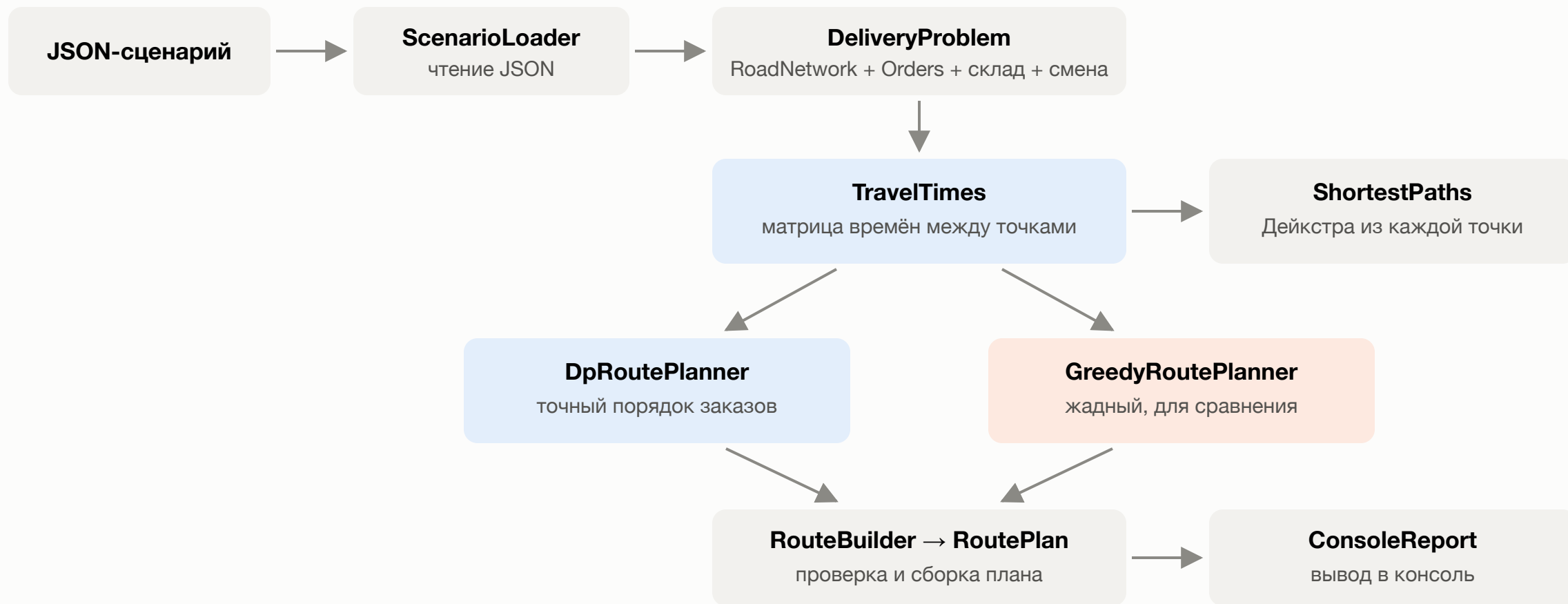
Три нумерации: перекрёсток, точка, заказ

ИНДЕКС	ЧТО НУМЕРУЕТ	ДИАПАЗОН	ГДЕ ИСПОЛЬЗУЕТСЯ
node — перекрёсток	Вершина графа города	0 ... V-1	<code>RoadNetwork</code> , Дейкстра
point — точка	Склад и адреса заказов	0 = склад, 1 ... k = заказы	<code>TravelTimes.Minutes(from, to)</code>
order — заказ	Элемент <code>problem.Orders</code>	0 ... k-1	DP, жадный, битовые маски

Заказ i = точка $i + 1$, склад = точка 0.
Отсюда все `i + 1` и `next + 1` в коде.

Точка → перекрёсток: через массив `_nodeOfPoint` в `TravelTimes` .

Как классы передают данные друг другу



RoadNetwork — граф города списком смежности

ЧЛЕН	ЧТО ЭТО
<code>_adjacency</code>	Список смежности: <code>_adjacency[v]</code> — все дороги из перекрёстка <code>v</code>
<code>_names</code>	Имя по номеру: <code>_names[3]</code> = «Вокзал»
<code>_indexByName</code>	Обратный словарь: номер по имени
<code>NodeCount</code> / <code>RoadCount</code>	<code>V</code> и <code>E</code> ; двусторонняя дорога = 2 дуги
<code>AddNode(name)</code>	Добавить перекрёсток или вернуть номер существующего
<code>AddRoad(from, to, minutes, twoWay)</code>	Добавить дорогу. Здесь отклоняется отрицательное время
<code>RoadsFrom(v)</code>	Дороги из <code>v</code> — их перебирает Дейкстра
<code>Name(v)</code> / <code>IndexOf(name)</code>	Перевод номер ↔ имя

Road — одна дуга

`To` — номер перекрёстка, куда ведёт дорога

`Minutes` — время проезда

Внутри — только числовые индексы.

Имена нужны лишь на входе и при выводе.

Dijkstra(network, source) — переменные

ПЕРЕМЕННАЯ	ЧТО ЭТО
<code>n</code>	Число перекрёстков
<code>distance[v]</code>	Лучшее известное время от source до v; в начале <code>Unreachable</code>
<code>previous[v]</code>	Откуда пришли в v по кратчайшему пути; -1 — у источника и недостижимых
<code>heap</code>	Куча (перекрёсток, расстояние): достаёт ближайший
<code>node</code> , <code>dist</code>	Извлечённый перекрёсток и расстояние, с которым его положили
<code>dist > distance[node]</code>	Устаревшая запись — пропуск (ленивое удаление)
<code>candidate</code>	Время до соседа через node; в <code>long</code> — без переполнения

ShortestPathTree — результат

- `DistanceTo(v)` — кратчайшее время
- `CanReach(v)` — достигим ли v
- `PathTo(v)` — путь: идёт по `previous` от v назад к источнику, затем разворачивает список

В `PriorityQueue` нет `decrease-key` — поэтому вершина кладётся повторно, а старые записи отбрасываются.

Данные задачи и результат

Order — заказ

- `Id` — номер («A», «F1»)
- `Location` — имя перекрёстка-адреса
- `Value` — ценность, ≥ 1
- `DeadlineMinute` — крайняя минута прибытия; `null` — без дедлайна
- `ServiceMinutes` — время передачи

DeliveryProblem

- `Network` — город
- `Depot` — имя склада
- `Orders` — заказы
- `ShiftMinutes` — `T`, длина смены

Конструктор проверяет весь

ввод: адреса, повтор `Id`, отрицательные значения

RoutePlan — результат

- `Stops` — остановки (заказ, прибытие, отъезд)
- `Skipped` — недоставленные + причина
- `StreetPath` — путь склад $\rightarrow \dots \rightarrow$ склад
- `ReturnMinute`, `TotalValue`

SkipReason — почему заказ не доставлен

`Unreachable` — нет пути туда или обратно

`DeadlineUnreachable` — не успеть даже напрямую

`ShiftTooShort` — один не влезает в смену

`NotSelected` — не поместился в маршрут

TravelTimes — город сжимается до $k + 1$ точек

ЧЛЕН	ЧТО ЭТО
<code>_nodeOfPoint[p]</code>	Перекрёсток точки p: 0 — склад, $i+1$ — адрес заказа i
<code>_treeOfPoint[p]</code>	Дерево кратчайших путей Дейкстры из точки p
<code>treeByNode</code>	Кэш: один перекрёсток — один запуск Дейкстры
<code>PointCount</code>	$k + 1$
<code>DijkstraRuns</code>	Сколько раз реально запускалась Дейкстра
<code>Minutes(from, to)</code>	Главный метод: кратчайшее время между точками
<code>Path(from, to)</code>	Перекрёстки пути — для <code>StreetPath</code>

```
// TravelTimes: точка → перекрёсток → время
public int Minutes(int from, int to) =>
    _treeOfPoint[from]
        .DistanceTo(_nodeOfPoint[to]);

// после этого шага граф
// планировщику больше не нужен
```

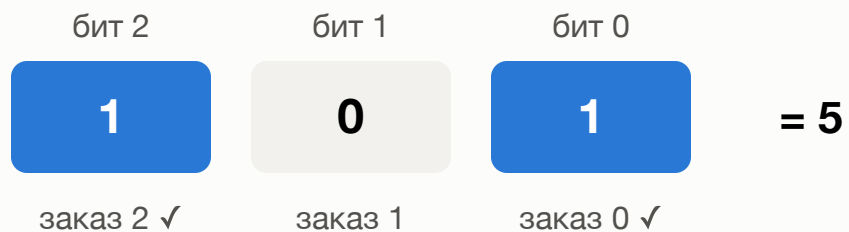
DP: константы и подготовка

КОНСТАНТА	ЧТО ЭТО
<code>MaxOrders = 20</code>	Максимум заказов: дальше таблица не помещается в память
<code>NotReached</code>	В <code>finish</code> : состояние не достигнуто (<code>int.MaxValue</code>)
<code>FromDepot = -1</code>	В <code>previous</code> : заказ был первым, перед ним — склад

ПЕРЕМЕННАЯ	ЧТО ЭТО
<code>k</code>	Число заказов
<code>deadline[i]</code>	Дедлайн заказа <i>i</i> ; без него — <code>long.MaxValue</code>
<code>service[i]</code>	Время передачи заказа <i>i</i>
<code>back[i]</code>	Кратчайшее время от заказа <i>i</i> до склада
<code>shift</code>	Длина смены <i>T</i>
<code>states</code>	Размер таблицы: $2^k \cdot k$

mask, last и таблица finish

Битовая маска доставленных заказов



- `mask` — какие заказы уже доставлены
- `last` — заказ, доставленный последним: у него стоит курьер
- `next` — заказ, который пробуем следующим

`finish[mask * k + last]`

Самая ранняя минута, когда курьер закончил заказ `last`, доставив ровно `mask`

`previous[mask * k + last]`

Какой заказ был перед `last`; `FromDepot` — если `last` первый.
Тип `sbyte` — 1 байт

Таблица двумерная `[mask][last]`, но хранится одним массивом: индекс = `mask · k + last`

Relax — один переход DP

ПЕРЕМЕННАЯ	ЧТО ЭТО
<code>fromPoint</code>	Откуда едем — точка (0 или last + 1)
<code>fromOrder</code>	Откуда едем — заказ , для previous
<code>time</code>	Когда освободились на прошлом месте
<code>leg</code>	Время проезда до next
<code>arrival</code>	Время прибытия к next
<code>done</code>	Конец передачи заказа next
<code>index</code>	Ячейка состояния (mask ∪ {next}, next)

```

int leg = travel.Minutes(fromPoint, next + 1);
long arrival = time + leg;
if (arrival > deadline[next]) return; // дедлайн

long done = arrival + service[next];
if (done + back[next] > shift) // отсечение:
    return; // не успеть назад

// добавили next в маску
int index = (mask | (1 << next)) * k + next;
if (done < finish[index]) // раньше?
{
    finish[index] = (int)done;
    previous[index] = fromOrder;
}

```

Выбор лучшего состояния и восстановление пути

ПЕРЕМЕННАЯ	ЧТО ЭТО
<code>valueOfMask[mask]</code>	Ценность заказов маски: <code>valueOfMask[mask & (mask-1)]</code> + ценность младшего заказа
<code>low</code>	Номер младшего бита (<code>TrailingZeroCount</code>)
<code>t</code>	finish текущего состояния
<code>returnMinute</code>	Возврат на склад: <code>t + back[last]</code>
<code>bestValue</code> , <code>bestReturn</code>	Лучшая ценность и возврат при ней; старт 0 и 0 — пустой маршрут
<code>bestMask</code> , <code>bestLast</code>	Где достигнут лучший результат; -1 — пустой маршрут

Восстановление — с конца

```
for (mask = bestMask, last = bestLast;
    last != FromDepot;)
{
    sequence.Add(last);
    int prev = previous[mask * k + last];
    mask &= ~(1 << last); // снять бит
    last = prev;
}
sequence.Reverse();
```

`mask & (mask - 1)` убирает младший установленный бит: 101 → 100

Жадный алгоритм и сборка плана

GreedyRoutePlanner — ближайший сосед

<code>done[i]</code>	Заказ <i>i</i> уже доставлен
<code>sequence</code>	Порядок выбранных заказов
<code>time</code> , <code>point</code>	Текущая минута и точка
<code>best</code>	Ближайший выполнимый заказ; <code>-1</code> — конец
<code>bestLeg</code>	Время езды до него
<code>leg</code> , <code>back</code> , <code>arrival</code>	Как в DP

RouteBuilder — общий для обоих

`TryBuild(problem, travel, sequence)` пересчитывает время по готовому порядку, проверяет дедлайны и смену, возвращает план или `null`

<code>orderIndex</code> / <code>next</code>	Номер заказа / его точка (+1)
<code>point</code> , <code>time</code>	Где курьер и который час
<code>BuildStreetPath</code>	Склеивает пути; <code>Skip(1)</code> убирает повтор стыка
<code>ClassifySkipped</code>	Причины для недоставленных

Интерфейс, загрузка сценария и консоль

IRoutePlanner

Общий интерфейс: Name и
Plan(problem, travel).DP и
жадный взаимозаменяемы

ScenarioLoader

JSON → DeliveryProblem.
ScenarioDto, RoadDto, OrderDto
повторяют структуру файла. Битый
JSON → FormatException

Program

Без аргументов — все сценарии,
путь к файлу — один сценарий,
--bench — замер

ConsoleReport

Печать задачи, планов DP и жадного и их сравнения.
Solve — решить и напечатать

Benchmark

Город-решётка 40×40, замер при k = 4...20. Side —
сторона решётки, dijkstraMs и dpMs — время шагов

Тестовые классы

КЛАСС	ЧТО ПРОВЕРЯЕТ
Reference	Независимый эталон: <code>FloydWarshall</code> и полный перебор <code>BestByBruteForce</code> . <code>AssertPlanIsValid</code> перепроверяет любой план с нуля
TestData	Помощники: короткое создание сети, случайные задачи <code>RandomProblem</code>
ShortestPathsTests	Дейкстра: обходы, недостижимость, одностороннее движение, сверка с Флойдом
DpRoutePlannerTests	DP: основной сценарий, пограничные случаи, сверка с перебором на 1500 задачах
GreedyVsDpTests	Жадный хуже DP на контрпримерах и никогда не лучше на случайных
ValidationTests	Некорректный ввод отклоняется
ScenarioFiles	Все демо-сценарии загружаются и решаются оптимально

Как DP проходит ловушку шаг за шагом

Заказы: 0 = N (Киоск), 1 = F1, 2 = F2. Смена 75. Со склада: N — 5, F1 — 30, F2 — 35 мин

МАСКА	LAST	FINISH	КАК ПОЛУЧЕНО
001 {N}	N	5	склад → N
010 {F1}	F1	30	склад → F1
100 {F2}	F2	35	склад → F2
011 {N, F1}	F1	40	$N(5) + 35$; возврат $40 + 30 = 70 \leq 75$ ✓
011 {N, F1}	N	65	$F1(30) + 35$; возврат $65 + 5 = 70$ ✓
101 {N, F2}	—	—	оба порядка дают возврат $80 > 75$ — отсечено
110 {F1, F2}	F2	35	$F1(30) + 5$; возврат $35 + 35 = 70$ ✓
110 {F1, F2}	F1	40	$F2(35) + 5$; возврат $40 + 30 = 70$ ✓
111 {все}	—	—	любой порядок даёт возврат > 75 — отсечено

Ценность: маска 011 → 6, маска 110 → **10**. Ответ — {F1, F2}, возврат в 70 мин